

WEPASStacks Inbound Congestion Prediction and Single-Drone Routing

An MLP-Filtered Ant Colony Optimisation Framework

Undergraduate Machine Learning Project Report

September 2026

Abstract

This project studies congestion prediction and drone pickup decisions using the WEPASStacks inbound-delivery data. Events from four inbound points are grouped into 15-minute intervals. A multilayer perceptron (MLP) uses seven current and recent features to produce one risk score for each warehouse. The target is positive when the warehouse is expected to reach 80% capacity within the next hour, assuming that no pickup occurs during that period.

Warehouses with goods are passed to ant colony optimisation (ACO) when their MLP score is at least 0.70. ACO then constructs a route for one simulated drone using pheromone and distance. On the held-out data, the MLP achieved an F1 score of 0.9699, compared with 0.8769 for the current-fill baseline. MLP-ACO reduced simulated overflow by 11.38% relative to ACO-current, but required more distance and energy. These results have two important limits: all positive validation and test labels belong to Warehouse 1, and no evaluated interval contains more than one active warehouse. The experiment therefore evaluates prediction and pickup timing, but not the quality of multi-stop ACO routing.

Keywords: warehouse congestion, multilayer perceptron, time-series classification, ant colony optimisation, drone routing, WEPASStacks

Contents

1	Introduction	3
2	Part A — Task Definition	3
2.1	Objective and Motivation	3
2.2	Project Scope and Data Assumptions	3
2.3	Training and Deployment I/O	4
2.4	Research Question and Success Criteria	5
3	Part B — Model, Algorithm, and System Structure	5
3.1	Dataset and Preprocessing	5
3.1.1	Dataset Structure and Quality	5
3.1.2	Aggregation and Zero Intervals	6
3.1.3	Rolling Features, Queue Simulation, and Leakage Control	6
3.2	End-to-End System Overview	6
3.3	MLP architecture	7
3.3.1	Validation comparison of model sizes	7

3.4	Loss, optimiser, and class imbalance	8
3.5	Framework and implementation choices	10
3.5.1	Alternative design choices	10
3.6	MLP filter and single-drone ACO	10
3.6.1	Deployment pseudocode	11
3.6.2	Computational size and feasibility	11
3.6.3	Synthetic multi-stop routing check	11
3.7	Theory-to-code mapping	12
3.8	Reproducibility and Hyperparameter Configuration	12
3.9	Implementation Details and Observed Challenges	13
4	Part C — Model Evaluation and Refinement	13
4.1	Expected Model Behaviour	13
4.2	Evaluation Protocol and Metrics	14
4.3	Prediction Performance	14
4.4	Operational Performance	16
4.5	Implementation Verification	16
4.6	Answer to the Research Question	16
4.7	Loss–Objective Mismatch	17
4.8	Interpretation and Trade-offs	17
4.9	Limitations	17
4.10	Recommended Refinements	17
5	Conclusion	18
6	Reproducibility Checklist	18
A	Configured Drone Experiment	18
B	Metric Definitions	19

1 Introduction

Inbound congestion occurs when goods arrive faster than they can be removed. A useful system should warn about current or near-term congestion and choose pickups that satisfy distance, energy, and payload limits. This project studies both tasks. The MLP estimates congestion risk for the current state and the next hour. ACO receives only the warehouses that pass the MLP filter and constructs a route for one drone.

A current-fill rule uses only the goods that are already waiting. It does not use recent arrival patterns and may therefore react late. The MLP is tested to determine whether recent arrivals provide useful warning information. It is compared with an always-negative rule and a current-fill rule. The routing policy is compared with nearest-first, fullest-first, and ACO-current. The MLP is deliberately small because the input is a short tabular feature vector.

The central research question is:

On the chronological held-out period, does filtering warehouses with an MLP risk threshold before single-drone ACO reduce overflow relative to current-fill routing, while respecting route-distance, energy, and payload constraints?

The prediction stage asks whether seven present-and-past features improve threshold classification over simple rules. The operational stage asks whether the resulting filter improves routing outcomes. A high classification score alone cannot answer the operational question.

2 Part A — Task Definition

2.1 Objective and Motivation

The proposed contribution is an MLP-filtered ACO pipeline for one drone. The MLP produces one score per warehouse. A hard threshold of 0.70 determines which non-empty warehouses are visible to ACO. The score is not reused as an urgency weight, projected-stock estimate, or ACO objective term. This separation makes the role of machine learning testable: it changes the candidate set, while ACO remains a distance-based route optimiser.

The report uses a drone-only queue definition. Goods leave a simulated warehouse only when the single drone collects them. Battery is restored at the beginning of every 15-minute interval, representing replacement or charging at the depot. The earlier service-rate and multi-drone designs are outside the scope of the current experiment. The training-state simulator and deployment simulator use the same warehouse capacities, single-drone configuration, per-interval battery policy, and queue-update equations.

2.2 Project Scope and Data Assumptions

The project distinguishes fields in the provided WEPAS stacks records from experimental assumptions introduced by the simulation. The supplied fields describe a temporal stream of inbound events; the current file does not include four geographically separated warehouses, drone specifications, a central depot, or measured congestion labels.

Table 1: Provided dataset information and simulated assumptions

Category		Used in the project	Interpretation
Provided	WEPAS-	Arrival time, inbound dock, SKU, batch, and week where available	The event records determine time-varying arrival demand in the experiment. The baseline model uses arrival time and dock; SKU and batch are excluded.
tacks fields			
Simulated	system	Queue length, historical drone pickups, warehouse capacities, and congestion at 80% capacity	These variables define a synthetic prediction target. They are not observed ground truth, and reproducibility must be confirmed from the final configuration and seed records.
state			
Simulated routing en-		Four satellite warehouses, planar coordinates, central depot, and one drone with payload, battery, energy, and route-distance limits	These values define the proposed routing experiment. One event is treated as one standardised cargo unit, not as a claim that a small drone carries a physical pallet.
vironment			

2.3 Training and Deployment I/O

Each training observation represents one dock at the end of a 15-minute interval. The input vector is

$$\mathbf{x}_{i,t} = [r_{i,t}, a_{i,t}, \bar{a}_{i,t}^{(60)}, d_{i,1}, d_{i,2}, d_{i,3}, d_{i,4}] \in \mathbb{R}^7, \quad (1)$$

where $r_{i,t}$ is the buffer fill ratio, $a_{i,t}$ is the current number of arrivals, $\bar{a}_{i,t}^{(60)}$ is the mean arrival count over the current and previous three intervals, and $d_{i,j}$ is a one-hot dock indicator. The feature definition is intended to restrict input values to information available at or before time t ; this property must be confirmed in the final leakage audit.

The binary training target assumes that no drone will collect goods during the next four intervals. It is

$$y_{i,t} = \mathbb{1} \left[\max_{h \in \{1,2,3,4\}} \left(A_{i,t} + \sum_{s=1}^h a_{i,t+s} \right) \geq 0.8K_i \right], \quad (2)$$

where $A_{i,t}$ is the available stock before the current pickup decision and K_i is warehouse capacity. The configured capacities are 200 units for Warehouse 1 and 80 units for Warehouses 2–4. Future arrivals are required for historical label construction. The feature-building procedure is intended to exclude them from the input vector, and that separation must be verified before final reporting.

Table 2: Technical input and output specification

Phase	Interface	Valid input	Output
Training-label construction	One warehouse–interval sample	A seven-value vector in the fixed order shown above; the three numeric features are scaled from training statistics and the four dock indicators remain binary	One binary ground-truth label indicating whether no-drone stock reaches 80% capacity during the next four intervals; 0.80 is not an MLP output threshold
MLP inference	One simultaneous four-warehouse state	A 4×7 float32 feature matrix containing current fill ratio, current arrivals, trailing four-interval mean, and dock identity	A vector $\hat{\mathbf{p}}_t \in [0, 1]^4$ containing one continuous sigmoid score per warehouse; the MLP does not output a fixed value of 0.80
Pre-ACO filter	Current goods and four MLP scores	Warehouse i is retained only if $A_{i,t} > 0$ and $\hat{p}_{i,t} \geq 0.70$	A sorted mapping from active warehouse identifier to currently available goods
ACO deployment	Filtered goods, a 5×5 distance matrix, one drone state, ACO parameters, and seed	Non-negative goods; valid depot/warehouse nodes; positive payload, energy, battery, and maximum-distance limits	One depot-to-depot route, proportional pickup amounts, route distance, energy used, remaining active goods, objective value, and validation status

2.4 Research Question and Success Criteria

Table 3: Questions and pre-declared evidence required to answer them

Question	Evidence	Success criterion
Prediction	PR–AUC, ROC–AUC, precision, recall, F1, confusion matrix, and per-warehouse support on chronological validation and test data	Compare with always-negative and current-fill baselines; do not generalise positive-class performance to a warehouse without positive support.
Primary routing	Overflow, collected units, energy, distance, sorties, active-warehouse visits, runtime, and independently recomputed constraint checks	Compare MLP–ACO with nearest-first, fullest-first, and ACO-current on identical held-out arrivals.
Stability	Repeated ACO runs on identical states and arrival streams	Use the ten configured seeds for stochastic policies and report mean and standard deviation; label any shorter run as diagnostic.

3 Part B — Model, Algorithm, and System Structure

3.1 Dataset and Preprocessing

3.1.1 Dataset Structure and Quality

The supplied WEPASStacks documentation describes 89 days of operations in a finished-goods warehouse and three source files: a layout file, an order-event file, and an initial-fill file. The current implementation does not consume all three files directly. It uses a cleaned CSV derived from delivery events at inbound points 1–4; retrieval events, outbound points, the original layout, and initial inventory are outside the present learning interface.

The cleaned dataset contains 205,258 inbound events from days 2–89. Its nine fields include arrival time, dock, SKU, batch, and several derived time fields. The preprocessing checks found no missing values, invalid dock identifiers, negative times, or fully duplicated rows. There are 4,763 repeated combinations of arrival time and dock. They are retained because different cargo events can arrive at the same dock in the same second.

The main data-quality concern is unequal dock coverage. Dock 1 contains 93.26% of all events, and Docks 2–4 disappear early in the timeline. After day 39, the file contains events only for Dock 1. The dataset does not explain why this happens.

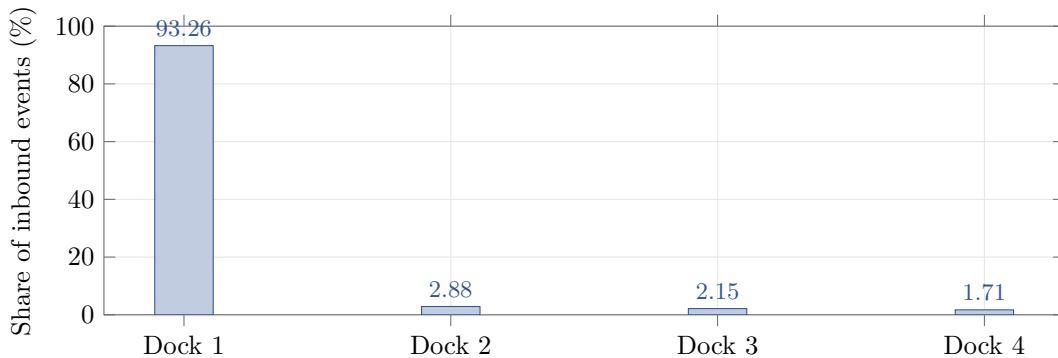


Figure 1: Inbound-event distribution by dock. Class weights may compensate for label imbalance, but they do not resolve unequal dock coverage.

Table 4: Temporal coverage and arrival distribution by dock

Dock	Events	Share	First day	Last day	Active days	Zero intervals
1	191,428	93.26%	2	89	88	20.25%
2	5,902	2.88%	2	39	12	92.65%
3	4,421	2.15%	2	18	8	94.78%
4	3,507	1.71%	2	11	5	95.83%

3.1.2 Aggregation and Zero Intervals

Arrival time is divided into 15-minute intervals, and events are counted for each dock. The pipeline then creates one row for every dock in every interval. This produces 33,580 dock–interval rows before incomplete rolling windows are removed. Missing event counts are filled with zero because an interval with no row may simply mean that no goods arrived.

This step also creates uncertainty. For Docks 2–4, a zero after the final recorded event may mean either no arrivals or that the dock was no longer recorded. The dataset has no `dock_active` field, so these cases cannot be separated.

3.1.3 Rolling Features, Queue Simulation, and Leakage Control

The 60-minute feature is the mean arrival count across the current and previous three intervals. The first three rows for each dock are removed because they do not have a complete window. A fixed fullest-first drone policy generates the historical queue states in time order. It uses only the information available at the current interval and does not use future arrivals or MLP predictions. The queue update is

$$A_{i,t} = Q_{i,t} + a_{i,t}, \quad (3)$$

$$R_{i,t} = \max(0, A_{i,t} - P_{i,t}), \quad (4)$$

$$O_{i,t} = \max(0, R_{i,t} - K_i), \quad (5)$$

$$Q_{i,t+1} = \min(K_i, R_{i,t}), \quad (6)$$

where $P_{i,t}$ is the quantity collected by the single drone. There is no separate service-rate term. Battery is reset before each simulated interval. The feature `buffer_fill_ratio` is calculated from $A_{i,t}$ before pickup, so it can exceed 1 when arrivals temporarily exceed capacity; its current modelling range is 0.20 to 1.55.

The final four rows for each dock are removed because their one-hour future target is incomplete. The final modelling table contains 33,552 observations. To reduce time leakage, the features use only current and past information, the scaler and class weights are fitted on the training set, and one-hour gaps are placed between data splits. Validation data is used during model development, while test data is reserved for final evaluation.

Table 5: Chronological data split

Split	Time range	Observations	Positive	Positive rate
Training	Days 2–60 with horizon gap	22,628	6,291	27.80%
Validation	Days 61–74 with horizon gap	5,360	965	18.00%
Test	Days 75–89	5,532	1,099	19.87%

3.2 End-to-End System Overview

Figure 2 shows the single-drone workflow. A fixed fullest-first policy produces historical queue states for supervised learning. During deployment, the MLP scores the four current states, the 0.70 filter removes low-score or empty warehouses, and ACO routes only over the retained set. Both stages use the same capacities, one-drone configuration, per-interval battery reset, and queue equations.

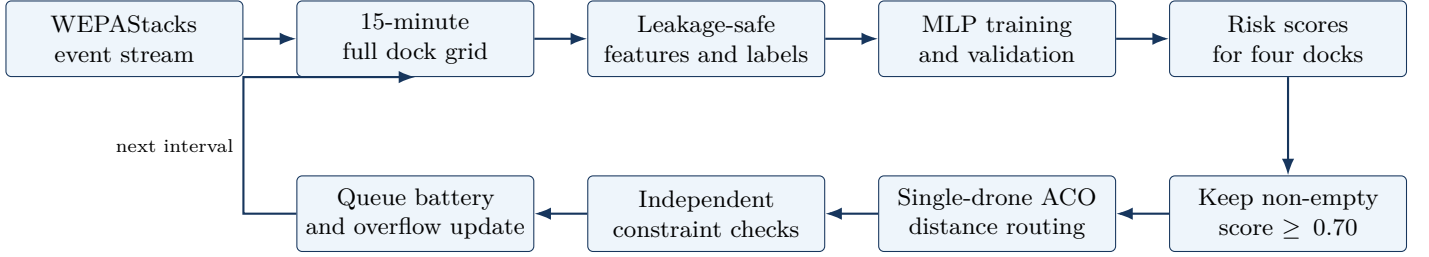


Figure 2: Updated data and decision flow. The MLP filters warehouses; its score is not used inside ACO.

3.3 MLP architecture

The network has two hidden layers. The first maps seven inputs to 32 ReLU units, and the second maps 32 values to 16 ReLU units. A single sigmoid output produces a risk score. Glorot uniform initialisation is used for the dense-layer weights, and all biases start at zero. No dropout layer is used in the drone-compatible implementation.

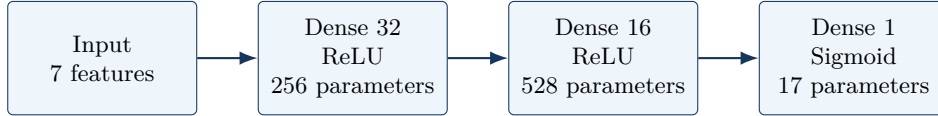


Figure 3: MLP architecture with 801 trainable parameters.

The forward pass can be written as

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad (7)$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2), \quad (8)$$

$$\hat{p} = \sigma(\mathbf{w}_3^\top \mathbf{h}_2 + b_3). \quad (9)$$

The two hidden layers allow the model to learn nonlinear relationships between current fill and recent arrivals. The complete network has only 801 trainable parameters, so it is a lightweight model for this tabular dataset. ReLU is used in the hidden layers, and the sigmoid output maps the final value to a score between zero and one [1, 2].

3.3.1 Validation comparison of model sizes

Three small architectures were trained with seeds 42, 43, and 44. Only validation data was used in this comparison. Table 6 reports the mean results. The 32–16 model obtained the highest mean F1 at the operational threshold. The 64–32 model produced almost the same PR–AUC but required more than three times as many parameters. The 32–16 model was therefore retained as a reasonable balance between validation performance and model size; the comparison does not prove that it is the only suitable architecture.

Table 6: Architecture comparison on validation data over three training seeds

Hidden units	Parameters	PR–AUC	F1 at 0.70	Mean best epoch
16	145	0.9908 ± 0.0008	0.9690 ± 0.0003	99.7
32–16	801	0.9917 ± 0.0002	0.9693 ± 0.0000	47.3
64–32	2,625	0.9917 ± 0.0003	0.9690 ± 0.0003	32.0

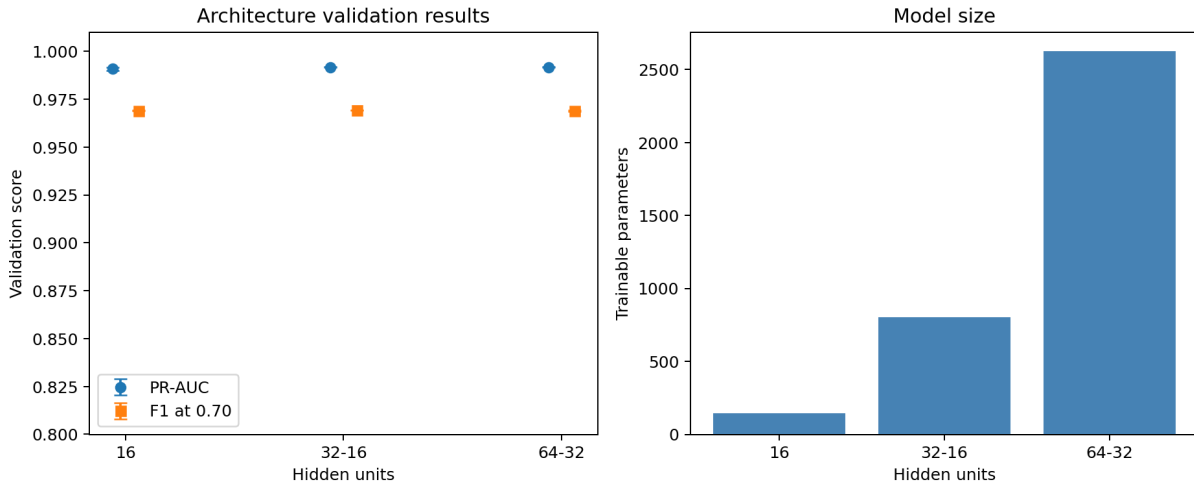


Figure 4: Validation performance and parameter count for the three candidate MLP architectures.

3.4 Loss, optimiser, and class imbalance

The network minimises weighted binary cross-entropy:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{n=1}^N w_{y_n} [y_n \log \hat{p}_n + (1 - y_n) \log(1 - \hat{p}_n)]. \quad (10)$$

The class weights are $w_0 = 0.6925$ and $w_1 = 1.7984$. They make mistakes on the less common positive class more costly during training. This helps with class imbalance but cannot fix the unequal data coverage between warehouses. Binary cross-entropy is used because the target is binary and the model has a sigmoid output. However, this training loss does not directly measure overflow, distance, or energy.

For each mini-batch, Keras calculates the forward pass and the weighted loss in Eq. (10). TensorFlow then calculates the gradients automatically, and the optimiser uses them to update the network weights. This means that backpropagation follows the stated loss even though the gradients are not coded by hand.

Adam with learning rate 10^{-3} was compared with SGD at learning rate 10^{-2} using the same 32–16 network, data split, and three seeds. Table 7 shows that Adam obtained the higher mean validation PR–AUC and F1 and reached its best validation loss earlier. SGD reached the 100-epoch limit in all three runs, which suggests that this SGD setting would require further learning-rate or momentum tuning. Adam was therefore retained for the final model [3]. This is a comparison of two specified settings, not proof that Adam is the best possible optimiser.

Table 7: Optimiser comparison on validation data over three training seeds

Optimiser	Learning rate	PR–AUC	F1 at 0.70	Best loss	Mean best epoch
Adam	0.001	0.9917 ± 0.0002	0.9693 ± 0.0000	0.0498	47.3
SGD	0.010	0.9898 ± 0.0000	0.9639 ± 0.0011	0.0616	100.0

Training uses batches of 256 and at most 100 epochs. Early stopping ends training when validation loss does not improve for eight epochs and restores the best weights. These values were chosen as reasonable baseline settings rather than through a complete hyperparameter search.

Figure 6 shows the training history of the final seed-42 model. Training stopped after 75 epochs, while the lowest validation loss occurred at epoch 67. Training and validation PR–AUC remained close, and validation loss did not show a large increase before stopping. This supports the use of early stopping for this split, but it does not solve the missing positive support at Warehouses 2–4.

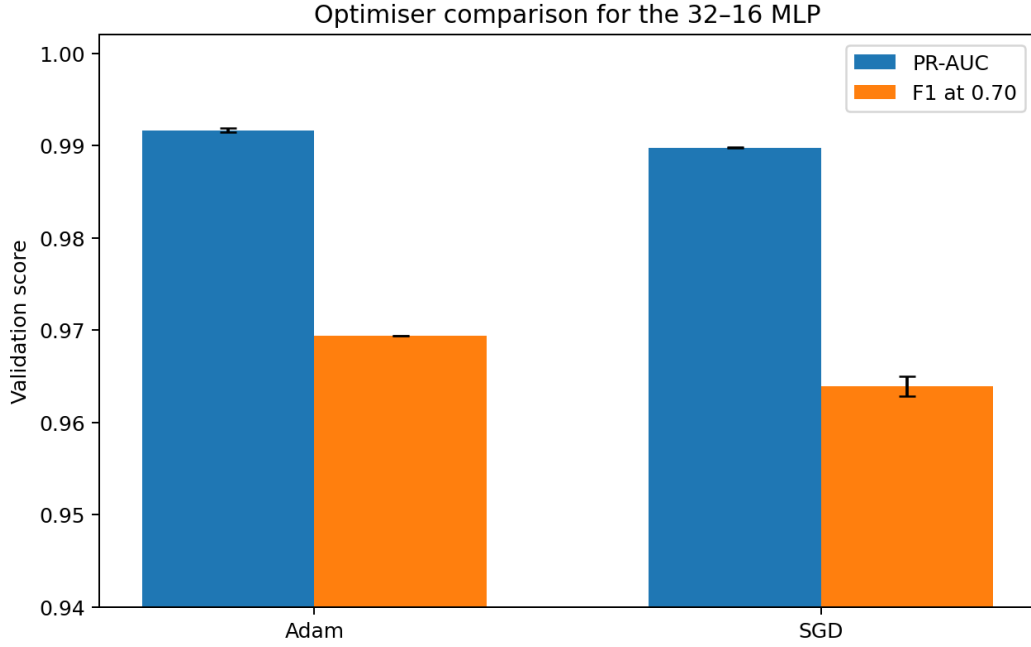


Figure 5: Validation PR-AUC and F1 for the tested Adam and SGD settings. Error bars show variation over three seeds.

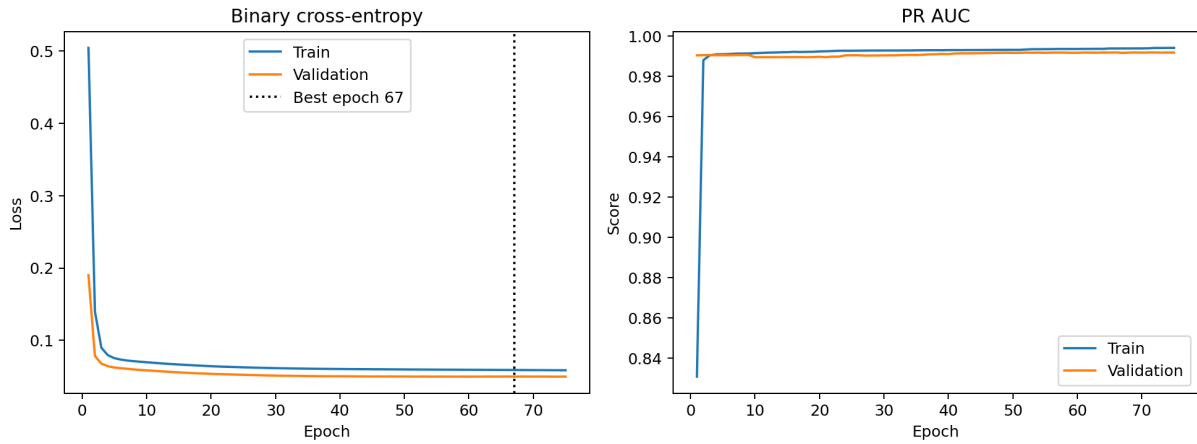


Figure 6: Training and validation history for the final 32-16 MLP. The vertical marker shows the epoch with the lowest validation loss.

3.5 Framework and implementation choices

pandas and NumPy are used for data cleaning, 15-minute aggregation, queue updates, and array calculations. scikit-learn provides feature scaling and evaluation metrics. TensorFlow/Keras provides the MLP layers, training process, early stopping, and model saving. During deployment, the saved scaler and feature order are loaded with the model so that training and inference use the same inputs.

ACO is implemented directly in Python so that its transition rule and route constraints are visible in the code. Route validation is kept separate from route construction. The validator recalculates payload, distance, energy, and warehouse eligibility instead of trusting the optimiser’s output. For only four warehouses, testing every possible route is also practical and provides a useful reference for ACO.

3.5.1 Alternative design choices

Table 8 summarises the main choices and their trade-offs. The alternatives are included to explain the design rather than to claim that every method was tested.

Table 8: Main design choices and alternatives

Choice	Reason used	Alternative	Trade-off
MLP	Learns nonlinear relationships between fill and recent arrivals	Logistic regression	Logistic regression is simpler; the measured performance difference is small.
Weighted BCE	Gives more importance to the less common positive class	Standard BCE	Standard BCE is simpler but may favour the majority class.
Hard 0.70 filter	Keeps prediction and routing roles easy to inspect	Put risk inside the ACO objective	A hard filter is clear but is sensitive to small score changes near 0.70.
ACO	Provides a routing method that can extend to more warehouses	Exact solver	Exact search guarantees the best small-instance route but scales poorly as the number of warehouses grows.
Adam	Performed better in the three-seed validation comparison	SGD	The tested SGD setting was slightly weaker and reached the epoch limit.

3.6 MLP filter and single-drone ACO

At interval t , available goods are $A_{i,t} = Q_{i,t} + a_{i,t}$. For MLP-ACO, the active set is

$$\mathcal{S}_t = \{i \mid A_{i,t} > 0 \wedge \hat{p}_{i,t} \geq 0.70\}. \quad (11)$$

Warehouses outside $\mathcal{S}_t$ are absent from the ACO input and cannot appear in its route. For nearest-first, fullest-first, and ACO-current, the active set instead uses the current-fill rule $A_{i,t}/K_i \geq 0.80$. This is the only difference between ACO-current and MLP-ACO; both use the same route optimiser.

Each ant begins at depot 0 and maintains the travelled distance. A candidate warehouse j is feasible only if the route can visit it and still return to the depot:

$$D_{\text{proj}} = D_{\text{so far}} + d_{ij} + d_{j0}, \quad (12)$$

$$D_{\text{proj}} \leq D_{\text{max}}, \quad (13)$$

$$D_{\text{proj}e} \leq B_{\text{available}}, \quad (14)$$

where e is energy per kilometre. Among feasible unvisited warehouses, the transition probability is

$$P(i \rightarrow j) = \frac{\tau_{ij}^\alpha (1/(\varepsilon + d_{ij}))^\beta}{\sum_{k \in \mathcal{F}} \tau_{ik}^\alpha (1/(\varepsilon + d_{ik}))^\beta}. \quad (15)$$

The MLP score and amount of goods do not appear in this transition rule. The implementation uses $\alpha = 1$, $\beta = 2$, 30 ants, at most 100 iterations, evaporation 0.10, and a 20-iteration stall limit.

For route R , the objective is

$$J(R) = 1000 |\mathcal{S}_t \setminus V(R)| + D(R), \quad (16)$$

where $V(R)$ is the set of visited active warehouses. Since every feasible route is limited to 30 km, the penalty makes an additional active visit more important than any possible distance saving. Among routes visiting the same number of active warehouses, the shorter route wins. After each iteration, pheromone evaporates and the global-best route receives an increment proportional to $1/(1 + J)$.

After route construction, the 30-unit payload is divided in proportion to available goods at visited warehouses. If $C = \min(30, \sum_{j \in V(R)} A_{j,t})$, then

$$x_{i,t} = C \frac{A_{i,t}}{\sum_{j \in V(R)} A_{j,t}}, \quad i \in V(R). \quad (17)$$

The validator recomputes depot endpoints, repeated and filtered nodes, total distance, route energy, payload, and per-warehouse pickup. Goods not collected remain in the queue. Battery is logged after the route and restored before the next interval.

3.6.1 Deployment pseudocode

Algorithm 1: MLP-filtered routing for one interval

Input: current queues, arrivals, saved scaler and MLP, drone and ACO settings.

Output: a validated route, pickups, updated queues, and interval metrics.

The procedure is:

1. Add the current arrivals to each warehouse queue.
2. Build the seven MLP features for all four warehouses.
3. Apply the saved scaler and MLP to obtain four risk scores.
4. Keep non-empty warehouses whose scores are at least 0.70.
5. Use ACO to construct feasible routes over the retained warehouses.
6. Select the route with the lowest unvisited-node penalty and distance.
7. Divide the payload in proportion to the goods at visited warehouses.
8. Validate the route, update the queues, and record overflow, distance, and energy.

This procedure matches the implementation order in `run_simulation()`. In particular, the MLP score is used for filtering and is not reused inside the ACO transition probability.

3.6.2 Computational size and feasibility

One MLP forward pass uses about $7 \times 32 + 32 \times 16 + 16 \times 1 = 752$ weight multiplications per warehouse, which is small for four warehouses. With m active warehouses, ACO performs approximately $O(IAm^2)$ candidate checks, where I is the number of iterations and A is the number of ants. Exact search is practical for the current four-warehouse experiment, whereas ACO is intended as a method that can be extended to larger routing problems.

3.6.3 Synthetic multi-stop routing check

The WEPAS stacks test stream never produces more than one active warehouse in an interval. To check the multi-stop code, 15 synthetic cases were created with two, three, or four active warehouses. ACO was run with ten seeds per case and compared with exact search.

Table 9: ACO compared with exact search on synthetic multi-stop cases

Active warehouses	Cases	ACO runs	Optimal found	ACO time (ms)	Exact time (ms)	
	2	5	50	100%	11.872	0.032
	3	5	50	100%	16.795	0.074
	4	5	50	100%	18.037	0.146

ACO matched the exact objective in all 150 runs and produced no constraint violations. This supports the implementation on the tested small cases, but it does not prove that ACO will be optimal on larger problems. Exact search was faster at the present scale.

3.7 Theory-to-code mapping

Table 10: Mapping from mathematical concepts to concrete implementation entry points

Component	Mathematical or system role	Function or class
Data preparation	Aggregates events, creates queue states, labels the target, and applies the chronological split	<code>aggregate_intervals()</code> , <code>simulate_historical_queue()</code> , and <code>chronological_masks()</code>
Forward and backward pass	Builds the $7 \rightarrow 32 \rightarrow 16 \rightarrow 1$ network; <code>model.fit()</code> invokes automatic differentiation and Adam on Eq. (10)	<code>_fit_mlp()</code> in <code>src/train_mlp_aco.py</code>
Prediction and filter	Restores the scaler, calculates risk scores, and applies Eq. (11)	<code>RiskPredictor</code> and <code>select_active_warehouses()</code>
ACO transition	Builds feasible routes using Eq. (15); the outer routine updates pheromone and tracks the best solution	<code>_construct_ant_solution()</code> and <code>optimize_aco()</code> in <code>src/aco_optimizer.py</code>
Objective and pickup	Implements Eq. (16) and proportional payload allocation	<code>calculate_route_objective()</code> and <code>allocate_pickups()</code> in <code>src/routing_utils.py</code>
Independent validation	Recomputes payload, battery, route distance, visited-node, and demand checks	<code>validate_solution()</code> in <code>src/solution_validator.py</code>
Rolling deployment	Calls prediction, filtering, routing, queue updates, overflow accounting, and per-interval battery reset	<code>run_simulation()</code> in <code>src/rolling_simulation.py</code>
Design checks	Runs the architecture, optimiser, and ACO-exact comparisons	<code>run_part_b_experiments()</code> in <code>src/part_b_experiments.py</code>

3.8 Reproducibility and Hyperparameter Configuration

The experiment uses seed 42 for Python, NumPy, and TensorFlow and requests deterministic TensorFlow operations where supported. The recorded environment uses Python 3.9.6, TensorFlow 2.20.0, Keras 3.10.0, NumPy 2.0.2, pandas 2.3.3, and scikit-learn 1.6.1. Small differences may occur on another computer or library version.

Table 11: Main MLP hyperparameters

Parameter	Value	Parameter	Value
Interval	15 minutes	Horizon	4 intervals
Capacities	200, 80, 80, 80	Congestion level	80% of capacity
Hidden units	32, 16	Trainable parameters	801
Optimiser	Adam	Learning rate	0.001
Batch size	256	Maximum epochs	100
Early-stopping patience	8	Operational threshold	Fixed at 0.70
Training boundary	Day 60	Validation boundary	Day 74
Validation F1 diagnostic	0.9313	Architecture seeds	42, 43, 44

These values describe the implemented baseline and are not the result of a large tuning study. The 60-minute horizon contains four 15-minute intervals. The time-based split prevents later observations from being used during training. The small validation studies support the 32–16 architecture and Adam setting among the tested candidates. The learning rate and batch size were not searched systematically. Held-out testing, a logistic-regression comparison, a warehouse-ID ablation, and repeated training seeds provide additional checks without proving that every hyperparameter is optimal.

3.9 Implementation Details and Observed Challenges

Preventing time leakage. Future arrivals are needed to create the target, but they must not appear in the seven input features. The data is therefore sorted by warehouse and time, rows with an incomplete future horizon are removed, and the scaler is fitted only on training rows. A chronological split is used because a random split could place later operating states in the training set.

Unequal warehouse coverage. Class weighting helps with the overall number of positive and negative labels, but it cannot create missing examples. Warehouse 1 dominates the data, while Warehouses 2–4 have no positive validation or test cases. The report therefore includes results by warehouse, a warehouse-ID ablation, and a logistic-regression baseline.

Hard decision threshold. Scores of 0.699 and 0.700 are almost equal, but only the second score passes the routing filter. Weighted BCE also trains the classifier without directly considering overflow, distance, or energy. The threshold experiment is therefore used to show trade-offs, not to claim that 0.70 is optimal. The output is called a risk score rather than an exact probability because its calibration is not perfect.

Limited ACO test cases. ACO can construct a route with several stops, but the WEPAS stacks evaluation data never has more than one active warehouse in an interval. The synthetic benchmark shows that the implementation matches exact search on the 15 tested small cases. However, these artificial cases do not replace operational evidence with real simultaneous demand.

4 Part C — Model Evaluation and Refinement

4.1 Expected Model Behaviour

The MLP should give a low score when current fill and recent arrivals are low. It should give a higher score when the warehouse is close to the congestion limit and arrivals remain high. Ideally, positive cases receive scores near one and negative cases receive scores near zero. Weighted BCE measures this behaviour during training, while held-out classification metrics evaluate it after training. Because some positive cases are already at 80% capacity, the final metrics include both current congestion detection and early forecasting.

The value 0.80 is used only to create the congestion label from warehouse capacity. The MLP produces a continuous score between zero and one. During deployment, a non-empty warehouse passes the filter when its score is at least 0.70. The best validation threshold is reported for comparison but does not replace the configured routing threshold.

The routing output should start and end at the depot and visit only warehouses that pass the filter. Each warehouse can be visited at most once. The route must remain within 30 km and 30 energy units, and the total pickup cannot exceed 30 cargo units. Prediction metrics evaluate the MLP, while overflow, collection, distance, energy, and constraint checks evaluate the routing policy.

4.2 Evaluation Protocol and Metrics

Accuracy alone can be misleading when the classes are imbalanced. Precision measures how many warnings are correct, recall measures how many positive cases are detected, and F1 balances both values. Balanced accuracy gives equal importance to the positive and negative classes. PR-AUC and ROC-AUC measure ranking performance across thresholds [5]. The confusion matrix and per-warehouse sample counts are also reported because a strong overall score may hide weak coverage at some warehouses.

Routing is evaluated using overflow, collected goods, distance, energy, number of sorties, active-warehouse visits, runtime, and constraint violations. MLP-ACO is compared with nearest-first, fullest-first, and ACO-current on the same held-out arrival stream. Each ACO policy is run with ten seeds.

4.3 Prediction Performance

The chronological split produced 22,628 training samples, 5,360 validation samples, and 5,532 test samples. Their positive rates were 27.80%, 18.00%, and 19.87%, respectively. Training stopped after 75 epochs. At the fixed operational threshold of 0.70, the held-out test confusion counts were $TN = 4418$, $FP = 15$, $FN = 50$, and $TP = 1049$. Table 12 reports the principal metrics.

Table 12: Held-out prediction results at threshold 0.70

Method	Bal. accuracy	Precision	Recall	F1	PR-AUC	ROC-AUC
Always negative	0.5000	0.0000	0.0000	0.0000	–	–
Current fill ≥ 0.80	0.8904	1.0000	0.7807	0.8769	–	–
MLP score ≥ 0.70	0.9756	0.9859	0.9545	0.9699	0.9938	0.9984

Across five training seeds, mean test PR-AUC was 0.99381 ± 0.00023 and mean F1 was 0.96976 ± 0.00068 . The small standard deviations show that training was stable for this data split. A feature-removal test was also performed without the warehouse-ID inputs. It slightly increased F1 from 0.9699 to 0.9704 but reduced PR-AUC from 0.9938 to 0.9826. Logistic regression achieved F1 0.9668 and PR-AUC 0.9938. The MLP therefore provides only a small improvement over the linear model.

The result is not equally supported across warehouses. All 1,099 positive test labels occur at Warehouse 1. Warehouses 2–4 each contribute 1,383 negative samples and no positive samples, so their recall, F1, and positive-class ranking performance cannot be estimated. Figure 7 makes this limitation explicit.

The Brier score was 0.0122. However, some middle score ranges contain few samples and do not follow the ideal line in the reliability diagram. For this reason, the output is described as a risk score rather than an exact probability. The validation threshold with the best F1 was 0.9313, while routing uses the fixed threshold of 0.70. The lower threshold allows the drone policy to react earlier.

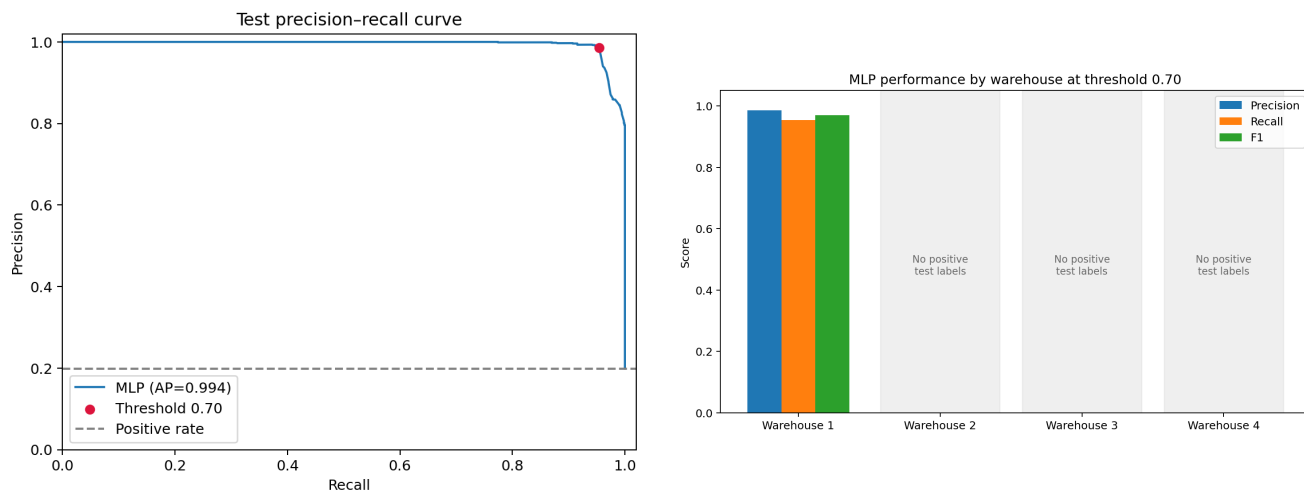


Figure 7: Held-out precision-recall evidence (left) and warehouse-specific support limitation (right).

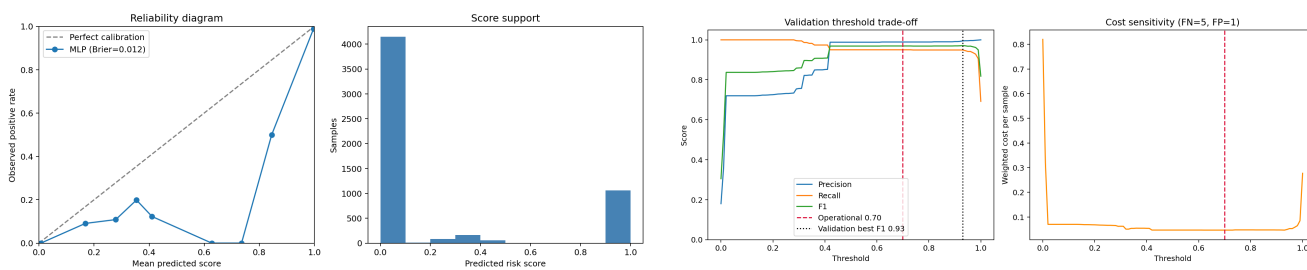


Figure 8: Reliability evidence (left) and validation threshold trade-off under an illustrative 5:1 false-negative cost (right).

4.4 Operational Performance

The complete held-out stream contains 1,387 time bins and 5,548 warehouse-interval rows. The ACO policies were evaluated with all ten configured seeds. At threshold 0.70, MLP-ACO reduced overflow from 6,219 to 5,511 units relative to ACO-current, a reduction of 11.38%. It also collected 738 more units. This improvement required 270 more distance and energy units. All checked payload, battery, distance, and route constraints were satisfied.

Table 13: Full-period operational comparison

Policy	Overflow	Collected	Distance	Energy	Sorties
Nearest-first	6219	25710	8570	8570	857
Fullest-first	6219	25710	8570	8570	857
ACO-current	6219	25710	8570	8570	857
MLP-ACO	5511	26448	8840	8840	884

No policy encountered more than one active warehouse in any interval. Therefore, every ACO seed produced the same totals. The current experiment can evaluate when the MLP triggers a pickup, but it cannot evaluate how well ACO orders several warehouse visits.

The threshold experiment shows a clear trade-off. Reducing the threshold from 0.70 to 0.30 lowered overflow from 5,511 to 4,445 units, but increased distance and energy from 8,840 to 10,620. A lower threshold protects the warehouses more aggressively but uses more resources. Selecting one threshold therefore requires a stated balance between overflow and operating cost.

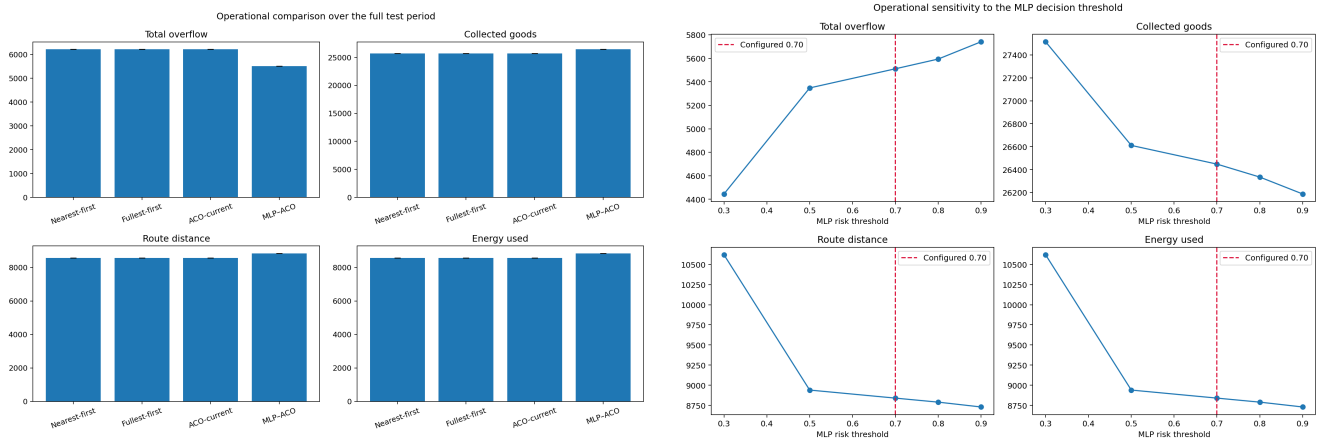


Figure 9: Full-period policy comparison (left) and downstream threshold sensitivity (right).

4.5 Implementation Verification

The test suite completed 22 tests with no failures. The tests cover the 0.70 boundary, empty and filtered warehouses, depot endpoints, repeated nodes, payload, energy, distance, battery reset, queue updates, fixed seeds, one exact-solver comparison, and reproducible generation of the synthetic routing cases. These tests support the checked cases but cannot cover every possible route or system state.

4.6 Answer to the Research Question

For Warehouse 1, the MLP improves F1 from 0.8769 for the current-fill baseline to 0.9699. MLP filtering also reduces simulated overflow by 11.38% relative to ACO-current. This supports the use of recent arrival information for earlier pickup decisions. The synthetic benchmark supports the multi-stop ACO implementation on small

artificial cases, but the WEPASacks result still does not show positive-case performance at Warehouses 2–4 or operational multi-stop routing.

4.7 Loss–Objective Mismatch

Weighted BCE measures prediction error, while the practical system also depends on overflow, distance, and energy. A score change has no routing effect if it remains below 0.70, whereas a change from 0.699 to 0.700 activates a warehouse. Similarly, the ACO objective rewards active visits and short routes but does not directly minimise overflow. For this reason, both prediction and operational metrics are reported.

4.8 Interpretation and Trade-offs

The fixed routing threshold is 0.70, while the best validation F1 occurred at 0.9313. Lower thresholds reduce overflow but increase distance and energy. The value 0.70 is therefore a configured operating choice, not a proven best threshold. The sigmoid output is called a risk score because it is not perfectly calibrated as a probability.

With only four warehouses, exact search is practical and provides a useful reference for ACO.

4.9 Limitations

1. **Simulated target.** Queue length and congestion are created from assumed capacities and drone behaviour, not measured warehouse congestion. Some positive cases are already at the 80% boundary, so the task mixes current detection with future forecasting.
2. **Unequal warehouse data.** Warehouse 1 contains 93.26% of inbound events. Warehouses 2–4 end early and have no positive validation or test labels.
3. **Data coverage and score interpretation.** For Warehouses 2–4, a zero may mean no arrivals or missing operation. Removing warehouse ID also changed the metrics, so the output is treated as a risk score rather than an exact probability.
4. **Limited feature set.** The baseline excludes hour-of-day, shift, robot state, downtime, service completions, and SKU mixture.
5. **Simplified drone model.** Battery is restored every 15 minutes, and the simulation omits flight time, charging time, weather, no-fly zones, and collision avoidance.
6. **Routing evidence.** The hard 0.70 filter can change a route after a small score change. The synthetic test checks multi-stop routing, but no WEPASacks interval contains more than one active warehouse.

4.10 Recommended Refinements

The first improvement is to distinguish an active dock with no arrivals from a dock that is inactive or not recorded. A `dock_active` field and an operating calendar would help. Real queue lengths, service completions, and confirmed capacities would also allow the simulated target in Eq. (2) to be compared with real congestion.

Future evaluation should keep chronological data splits and report the number of samples behind each metric. The model should also be evaluated only on cases with current fill below 0.80. This would test whether it predicts new congestion instead of only detecting warehouses that are already full. Any new calibration method should be fitted on validation data and tested on a new untouched period.

The routing threshold should be compared on a validation simulation before one value is selected. Future policies could also require a minimum pickup load. The ACO penalty could be changed to include expected overflow, although this would make the algorithm more complex.

Future operational routing tests need real or realistic time-series scenarios with several active warehouses at the same time. The completed synthetic benchmark should then be extended to larger networks where exact search becomes

difficult. Running more seeds on the current one-active-warehouse stream would not provide new route-order evidence.

5 Conclusion

The final system has two stages. First, the MLP produces one risk score for each warehouse, and a fixed 0.70 threshold selects non-empty warehouses. Second, single-drone ACO builds a distance-based route over the selected set. The route objective penalises unvisited active warehouses before distance, and the payload is shared in proportion to the available goods.

For Warehouse 1 and the simulated target, the MLP produced strong and stable prediction results. The 0.70 MLP-ACO policy reduced overflow by 11.38% relative to current-state routing, although it used more distance and energy. All checked constraints were satisfied. ACO also matched exact search in all 150 synthetic runs, but exact search was faster at this small scale. Warehouses 2–4 still have no positive test cases, and the operational stream does not test multi-stop routing. The results therefore support prediction, pickup filtering, and basic ACO correctness, but not a real multi-stop routing benefit.

6 Reproducibility Checklist

- Dataset file: `wepastacks_inbound_clean.csv`.
- Training recreated `model_outputs_single_drone`, including the saved model, predictions, metrics, calibration, threshold, seed-stability, ablation, and error-analysis artifacts.
- Prediction evidence includes baseline, support, threshold, confusion-matrix, and per-warehouse tables and figures.
- Command-line training entry point: `run_training.py`.
- Integrated experiment configuration: `config/experiment.yaml`.
- Routing entry point: `run_experiment.py`.
- Test command: `PYTHONPYCACHEPREFIX=/tmp/assignment2_pycache.venv/bin/python-munittestdiscover-tests-v`.
- Current verification: 22 tests passed in the available virtual environment; rerun immediately before submission.
- Part B experiment entry point: `run_part_b_experiments.py`; outputs include the architecture comparison, optimiser comparison, and synthetic ACO-exact benchmark.
- Routing recreated `outputs_single_drone` for the complete held-out period and all configured seeds.
- The run records ten seeds for each stochastic ACO policy and reports that the maximum simultaneous active-warehouse count is one.

A Configured Drone Experiment

Table 14: Simulated warehouse coordinates and capacities

Node	x (km)	y (km)	Capacity (cargo units)
Central depot 0	0	0	Not applicable
Warehouse 1	3	4	200
Warehouse 2	-5	2	80
Warehouse 3	6	-3	80
Warehouse 4	-4	-6	80

Table 15: Configured single simulated drone

Drone	Payload	Battery	Energy/km	Maximum route
1	30	30.0	1.0	30 km

Battery is reset before every 15-minute interval. This represents replacement or charging at the depot and is not an optimised charging schedule. The values in this appendix are experimental assumptions and must not be attributed to the WEPASStacks warehouse.

B Metric Definitions

For true positives TP , false positives FP , true negatives TN , and false negatives FN :

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad (18)$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \quad \text{F1} = \frac{2 \text{Precision Recall}}{\text{Precision} + \text{Recall}}, \quad (19)$$

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}. \quad (20)$$

The Brier score is the mean squared difference between a predicted score and the binary target. The held-out value is 0.0122. The accompanying reliability plot shows weak support and deviations in middle-score bins, so this value alone is not treated as proof of full calibration.

References

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- [2] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of AISTATS*, pages 249–256, 2010.
- [3] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- [4] M. Dorigo and L. M. Gambardella. Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1):53–66, 1997.
- [5] T. Saito and M. Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.
- [6] WEPASStacks project documentation. *Dataset Analysis Report, MLP Training Specification, MLP Training Report, Single-Drone MLP-ACO Implementation Guide, and Assessment Criteria*. Internal assignment documents, 2026.